Laborator 2 - Rolurile în echipă. Redactarea specificațiilor de proiect.

Rolurile în cadrul echipei de proiect

Cu cât proiectul este mai mare, cu atât rolurile din echipă sunt mai diversificate (mai specializate pe o anume arie de activități).

Într-un ***proiect mic***, principalele roluri din echipă sunt:

1. **Project Manager** / **Manager-ul Proiectului**
   - responsabil de succesul/eșecul proiectului
   - planifică & monitorizează evoluția proiectului
   - ia decizii
   - adesea, are background tehnic (pentru a înțelege ciclul de viață al proiectului)
   - întotdeauna, are abilități soft (de conducere, de motivare și stimulare, de conștientizare a cauzelor, de decizie, de negociere, de previziune, de inovație)
   - certificări PMI (Project Management Institute): PMP (Project Management Professional)
2. **Team Leader** / **Liderul echipei** (Dezvoltator Lider, QA Lider)
   - responsabil de execuția conform standardelor a activităților planificate
   - monitorizează activitatea echipei sale
   - intervine pentru a corecta, pentru a îndruma (coaching/support)
   - raportează managerului statusul real al proiectului
3. **Software Developer** / **Dezvoltator software**
   - responsabil de implementarea software a cerințelor
   - în metode de dezvolare agile contribuie și la design, arhitectură, specificații
4. **Tester** (Inginerul Calității)
   - responsabil de verificarea funcționalităților și a performanțelor produsului
   - scrie scenarii de test, le execută, analizează rezultatele ⇒ rapoarte de testare

Într-un ***proiect mare***, pe lângă rolurile de mai sus, apar și altele, precum:

1. **System Architect** / **Arhitectul Sistemului**
   - responsabil de arhitectura produsului software
   - captează interesele partenerilor de business și le transpune alegând arhitectura potrivită
2. **Technical Writer** / **Scriitorul tehnic**
   - responsabil de documentația tehnică a proiectului
   - primește documente tehnice de la ingineri, pe care le editează spre a fi corecte, clare și conforme cu standardele în vigoare
3. **Analyst** / **Analist** (Analist de Business, Analist de Sisteme Business, Analist de Sistem, Analist de Cerințe)
   - responsabil de înțelegerea corectă a cerințelor de business
   - studiază specificațiile clienților, clarifică toate detaliile proiectului cu clientul și cu partenerii (stakeholders)
   - redactează specificațiile aplicației software
4. **Consultant**
   - analizează fezabilitatea ofertelor de proiecte
   - propune soluții pentru diverse probleme legate de ciclul de viață al proiectelor
   - evaluează gradul de atingere a obiectivelor proiectelor
5. **Experți business**: SME (Subject Matters Expert), Manager-ul Produsului, Manager-ul de program

Gestiunea timpului

Orice membru al echipei și project manager-ul în special trebuie să aibă skill-uri de organizare a timpului pentru a obține maximum de rezultate. Aspecte precum planificarea task-urilor, stabilirea obiectivelor, evitarea întreruperilor, prioritizarea activităților trebuie avute constant în vedere pentru eficientizarea muncii.

Redactarea specificațiilor de proiect

După primirea specificațiilor clientului și după analiza de profunzime a acestora, contractorul redactează *documentele de specificație* ale soluției oferite, de obicei sub formă de:

- **Software Requirements Specification** / **Specificarea Cerințelor Software** (SRS/SCS)
- **Software Design Description** / **Descrierea Design-ului Software** (SDD/DDS)

Este **esențial** pentru reușita proiectului ca aceste documente de specificații să fie complete și absolut clare deoarece:

- acestea reprezintă parte a contractului dintre client și contractor;
- pe baza acestora se pot determina clar cerințele proiectului, rezolvându-se astfel eventualele conflicte dintre parteneri
- pe baza acestora se vor realiza estimările de timp și buget și planificările de proiect;
- pe baza lor se va realiza dezvoltarea soluției și testarea funcționalităților și performanțelor ei;
- pe baza lor se va realiza evaluarea finală a proiectului (a gradului de atingere a obiectivelor).

Documentul SRS cuprinde descrierea completă a comportamentului sistemului software, adică:

- a interacțiunilor dintre utilizator și sistem (**cerințe funcționale**). Acestea se descriu cu ajutorul *use-cases* (cazurilor de utilizare).
- a constrângerilor de proiectare și dezvoltare (**cerințe non-funcționale**). Acestea se referă la restricții de performanță, de securitate, de fiabilitate etc.

Conținutul unui SRS/SCS

- Scopul documentului (*Document purpose*)
- Conținutul documentului (*Document overview*)
- Descrierea generală a produsului (***General description of the product***)
- Situația curentă (*The current situation*)
- Misiunea proiectului (*Purpose of the product*)
- Contextul proiectului (*Product context*)
- Beneficii (*Benefit*)
- Cerințe funcționale (***Functional requirements***)
- Actori (***Actors***)
- Diagrama de sistem (***System boundary***)
- Descrierea cazurilor de utilizare (***Use cases description***)
- Cerințe nefuncționale (***Non-functional requirements***)
- Cerințe de interfață (*User Interface Requirements*)
- Cerințe de performanță (*Performance Requirements*)
- Cerințe de fiabilitate (*Availability & Reliability*)
- Cerințe de securitate (*Security Requirements*)

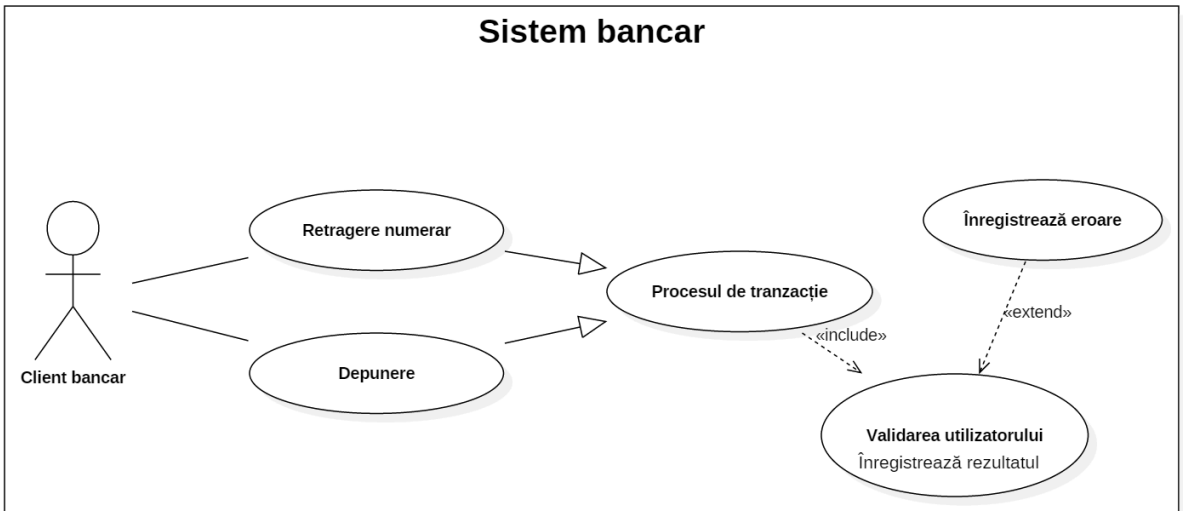
System Boundary

Se mai numește și *Use-case diagram / Diagrama de cazuri de utilizare*.

Este:

- o diagramă UML obținută în urma analizei cerințelor utilizatorului
- schema funcționalităților oferite de sistemul software, în termeni de actori, cazuri de utilizare și relații între acestea.

Mai multe detalii, poți afla aici [http://en.wikipedia.org/wiki/Use_case_diagram#Use_Case_Relationships] și aici [http://en.wikipedia.org/wiki/Use_case_diagram#Use_Case_Relationships].



Astfel de diagrame se pot realiza în Visio [http://office.microsoft.com/en-us/visio/], Visual Paradigm [https://www.visual-paradigm.com/download/community.jsp] sau Dia [https://wiki.gnome.org/action/show/Apps/Dia?action=show&redirect=Dia].

Exemplu de document SRS

Exemplu de Document SRS - Alumni database

Exemplu de Document SRS - Simulator cursa formula 1

Arhitectura proiectului

Stabilirea arhitecturii unui sistem software implică:

1. *alegerea șabloanelor arhitecturale* potrivite (se pot combina mai multe, se pot personaliza)
2. în funcție de șabloanele arhitecturale alese, *proiectarea structurii* sistemului software la nivel de:
    - **componente** specializate pe o gamă de servicii (module și interfețe),
    - **conexiuni** dintre componente.

Există mai multe tipuri șabloane arhitecturale (**architectural patterns**), precum:

- Tehnică de calcul distributivă [http://en.wikipedia.org/wiki/Distributed_computing]
    - sistemul software e alcătuit dintr-o serie de componente software ce rulează pe mașini de calcul diferite, ce comunică prin rețea
    - mașinile de calcul interacționează pentru a oferi cât mai rapid un răspuns corect la cererea utilizatorului
- Client-server [http://en.wikipedia.org/wiki/Client-server]
    - arhitectură distribuită
    - conține componente furnizoare de servicii = componente *server* și componente ce solicită servicii = componente *client*
- Arhitectură bazată pe evenimente [http://en.wikipedia.org/wiki/Event_Driven_Architecture]
    - se bazează pe producerea, detectarea, consumarea și reacția la evenimente
- Front-end și back-end [http://en.wikipedia.org/wiki/Front-end_and_back-end]
    - front-end = interfață de colectare a datelor de la utilizator și de procesare a acestora pentru a fi aduse la formatul prevăzut de componenta back-end
    - back-end = componenta software de procesare a datelor furnizate de front-end
- Modelul pe trei nivele [http://en.wikipedia.org/wiki/Three-tier_(computing)]
    - arhitectură de tip client-server
    - conține: nivel de prezentare (browser web), nivel de logică business (middleware - server de application: “motor” de rulare a paginilor web dinamice scrise în ASP.NET, JSP/Java, PHP, Perl, Python, Ruby), nivel de bază de date (server de bază de date)
    - browserul trimite cererile către serverul de aplicație, care le servește interogând și modificând conținutul bazei de date și, de asemenea, generează pagina de răspuns care va fi trimisă înapoi browserului.

Exemplu de descriere a arhitecturii unei aplicații: Platformă de gestiune a datelor medicale electronice

Software Design Document (SDD)

- document de **specificație** a soluției pentru sistemul software descris în **SRS**
- răspunde la întrebarea: *cum va fi construit sistemul software pentru a avea comportamentul descris în SRS?*
- prezinta metodologii, tehnologii, participanți și resurse implicate în proiect
- se poate redacta numai după finalizarea SRS-ului fiind un răspuns la cerințele prezentate în SRS
- este redactat de o echipă de software designers (proiectanți de sistem) și analiști business, pe baza SRS-ului și a experienței acestora
- reprezintă ghidul de construire a soluției folosit de echipa de dezvoltare a proiectului
- reprezintă un tool de analiză a întregului proiect în faza de început cât și de monitorizare pe parcurs

Secțiuni SDD

Un SDD are următoarele secțiuni:

1. **Scopul documentului (Document purpose)**
2. **Obiective (Objectives)**
3. **Conținutul documentului (Document overview)**
4. **Modelul datelor** (*Data Design*)
    - prezintă *structurile de date* importante, *formatele fișierelor* folosite în cadrul soluției și *schema bazei de date*.
    - Structurile de date pot fi:
        - **globale** (structuri de date disponibile tuturor componentelor arhitecturii)
        - **de legătură** (structuri de date trimise ca argumente între componente pentru a asigura fluxul informației la nivel de aplicație)
        - **temporare** (structuri de date folosite temporar).
    - Schema bazei de date este descrisă prin:
        - **diagrama schemei** bazei de date
        - **descrierea semnificației tabelelor** și, pentru fiecare tabelă, descrierea semnificației coloanelor și indicarea cheilor primare și referențiale.
5. **Modelul arhitectural** (*Architectural Design*)
    - este o structură ierarhică alcătuită din componente interconectate
    - prezintă arhitectura sistemului – atât descriptiv, cât și sub forma unei diagrame de arhitectură
    - descrie:
        - fiecare componentă a arhitecturii,
        - restricțiile de implementare ale componentelor,
        - interacțiunea dintre componentele sistemului.
    - poate fi reprezentat prin :
        - **diagrame de componente** (pentru proiecte mari) - le-am numit “generic” în laboratorul 2: diagrame de arhitectură
        - **diagrame de clase**, în care relațiile ierarhice se bazează pe generalizare și specializare (pentru proiecte mici).
6. **Modelul interfeței cu utilizatorul** (*User Interface Design*)
    - prezintă *ferestrele* aplicației și *succesiunea* lor în cadrul sistemului.
7. **Elementele de testare** (*Testing Issues*)
    - identifică *componentele critice* ale aplicației (componente a căror performanță influențează decisiv performanța globală a aplicației)
    - propune *alternative* de proiectare a componentelor critice (pentru a fi folosite în cazul insuccesului soluției propuse inițial).

Exemplu de Document SDD

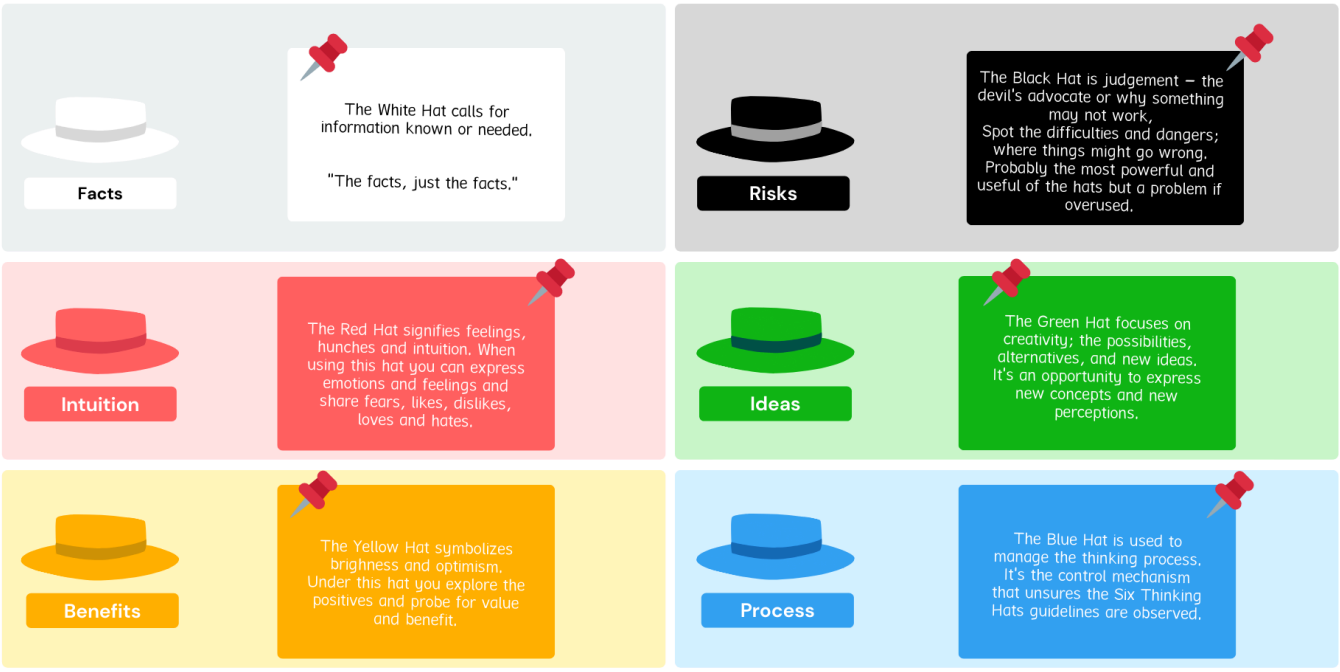
Exemple de document SDD: aici [https://drive.google.com/drive/folders/1rCzM9BKi24f7W3UOvrH1LRrUy6ZN1Kb-?usp=sharing]

Exerciții

Metoda pălăriilor gânditoare (30 - 40 minute)

Asistentul va propune câte o situație pentru fiecare persoană. Abordați problema dată din toate cele 6 perspective și apoi prezentați colegilor rezultatul.

Puteți folosi acest template [https://docs.google.com/presentation/d/15tcRtCq3ssXOIRSzIgdLHO1deBdqP_lSfT_BC6ZoUwk/edit?usp=sharing] sau acesta [https://online.visual-paradigm.com/app/diagrams/#diagram:proj=0&type=SixThinkingHats&gallery=/repository/fe32a848-e3e8-4c18-bbe2-1da5070521eb.xml&name=Six%20Thinking%20Hats%20Template] pentru a nota punctele reieșite din discuție.



Mai multe informații despre Metoda pălăriilor gânditoare aici [https://www.lucidchart.com/blog/about-the-six-thinking-hats-technique] și aici [http://www.debonogroup.com/six_thinking_hats.php].

Team Roles Test (15 minute)

Într-o clasificare de roluri pe echipă apar umătoarele, descrise mai jos.

Președinte: *Președintele are un rol puternic de coordonator. Punând accent pe proceduri, președintele va încerca să aducă laolaltă și să păstreze echipa împreună. El sau ea comunică și se ocupă de membrii echipei într-un mod respectuos și deschis.*

Expert: *Expertul are abilitățile și expertiza necesare pentru sarcina specifică la îndemână. El sau ea se concentrează puternic pe sarcină și poate intra într-o atitudine defensivă atunci când alții interferă cu munca lui sau ei. Expertul preferă să lucreze singur, iar membrii echipei dețin adesea o mare încredere în acesta/aceasta.*

Executiv: *Executivul este uneori menționat și ca organizator. Executivul este în general disciplinat și dornic să-și facă treaba. Este eficient, practic și sistematic. Executivii sunt bine organizați și harnici și transformă rapid ideile unei echipe în acțiuni concrete și planuri practice.*

Supraveghetor: *Supraveghetorul este în general foarte ambițios și energic. El sau ea poate să pară nerăbdător(oare) și impulsiv(ă). Supraveghetorul este un motivator puternic și îi va provoca pe alții în momentele vitale. Cu toate că acțiunile acestuia pot părea uneori oarecum emoționale, ele joacă un rol crucial în conducerea echipei spre reușită.*

Cercetător: *Cercetătorul este, în general, un extrovertit prin natură. El sau ea este vesel(ă), gregar(ă). Cercetătorul este, de asemenea, un bun investigator, interesat și curios despre lucruri. Deoarece doresc să improvizeze și să comunice cu alții, vor avea o mică problemă în a prezenta idei echipei și în a dezvolta noi contacte.*

Analist: *Analistul are tendința de a fi rezervat și critic. Analistul va reacționa, de asemenea, la planuri și idei într-un mod rațional și sensibil. El sau ea va favoriza o abordare prudentă a problemelor și le va evalua în funcție de exactitatea lor înainte de a acționa.*

Integrator: *Integratorul este foarte conștiincios și se simte responsabil pentru realizările echipei. Integratorii sunt preocupați de erori și au tendința de a-și face multe griji din cauza naturii lor de control. Integratorul este, de asemenea, cunoscut sub denumirea de finisor, deoarece este folosit cel mai eficient la sfârșitul unei sarcini, pentru a lustrui și a controla munca pentru erori, supunându-l la cele mai înalte standarde de control al calității.*

Inovator: *Inovatorul este adesea generatorul creativ al unei echipe. Are o imaginație puternică și o dorință de a fi original. Inovatorul preferă să fie independent și tinde să abordeze sarcinile într-un mod științific. Ca individ creativ, inovatorul poate juca un rol crucial în modul în care o echipă abordează sarcini și rezolvă probleme.*

Jucător de echipă: *Jucătorul echipei e grijuliu, evită conflictele și promovează armonie. Fiind cineva căruia îi place să ajute alte persoane, jucătorul echipei este în general considerat agreabil și prietenos. El sau ea este diplomatic și subliniază solidaritatea și coeziunea echipei.*

Fiecare dintre noi are ponderi pentru fiecare rol. Faceți o estimare a rolului pentru care considerați că aveți cea mai mare pondere. Apoi faceți o estimare a rolului fiecărui membru al echipei din care faceți parte (fiecare face estimarea proprie).

Apoi faceți fiecare testul de aici [https://www.123test.com/team-roles-test/index.php]. Vedeți ce rezultate obțineți atât voi cât și restul echipei și vedeți cât de bine ați estimat rolul vostru și al fiecărui membru al echipei.

Lucru la proiect (35-40 minute)

Echipele vor fi formate din 5-6 studenți. Apoi urmează stabilirea unui manager de proiect și a celorlalte roluri pe baza testelor efectuate mai sus (1 x Project Manager, 1x Team Leader, 1 x Tester, 1-3 x Dezvoltatori)

În echipă gândiți-vă cum veți aborda proiectul. Puteți folosi hârtie fizică, un fișier .txt sau .doc/.odt, un document Google sau o pagină wiki sau puteți trimite pe e-mail primele idei. Urmăriți:

- Ce soluții software vă propuneți să utilizați?
- Ce arhitectură de proiect urmăriți (în linii mari)?
- Care sunt responsabilitățile macro în cadrul echipei?
- Care este planul pentru următoarea săptămână (ce aveți de făcut fiecare)?
- Care este agenda următoarei întâlniri de proiect (din laboratorul următor)? Dau dacă planificați o întâlnire înainte de următorul laborator.
- Ce veți folosi pentru comunicare/colaborare?
    - Un director Google Drive [https://drive.google.com] în care veți pune documente de organizare. Va fi partajat cu asistentul.
    - O listă de discuții, un grup de Facebook sau ce doriți voi pentru comunicarea în cadrul echipei.

Să puneți la punct de pe acum resursele pe care le veți folosi.

Nu vă recomandăm să vă gândiți **acum** la un timeline de lucru la proiect, dar ar trebui să-l aveți în vedere în următoarele zile ce să știți:

- ce aveți de făcut
- cine ce face
- care este termenul pentru fiecare acțiune

Bonus: Testul culorilor

Testul culorilor [https://my-personality-test.com/true-Colours] este un test care prezintă personalitatea cuiva pe 4 culori:

- Mov - Loial, Demn de încredere, Pregătit, Organizat, Îngrijitor
- Portocaliu - Fermecător, Spontan, Are impact, Optimist, Îndrăzneț, Definitoriu
- Albastru - Flexibil, Arătos, Călduros, Milos, Imaginativ
- Verde - Analitic, Relaxat, Inventiv, Rațional, Teoretic

Pentru fiecare culoare de mai sus estimați ce scor veți obține între 0 și 100.

După ce faceți estimarea faceți și testul. Comparați estimarile si rezultatele.

mps/laboratoare/laborator-02.txt · Last modified: 2024/10/17 16:17 by iulia.stanica